



PRINICPLES OF COMPILER DESIGN

(Assignment_01)

NAME: Dagmawi Taye

ID:BDU1505627

Sec:A

Q=#7

Q#7 (Theory) : Define left recursion and explain why it is a problem in parsing.

(C++): Write a C++ program to tokenize arithmetic expressions with integers and +/*.

(Problem-solving): Grammar:

$S \rightarrow AB$

$A \rightarrow aA \mid \epsilon$

$B \rightarrow bB \mid b$

Construct the parse tree for "aab"

1. Theory: Left Recursion in Parsing

1.1 Definition

Left recursion occurs in a context-free grammar when a non-terminal can derive a string that begins with the same non-terminal. It is classified into two types:

1. Direct Left Recursion

A production of the form

$A \rightarrow A\alpha \mid \beta$

where α and β are sequences of terminals and/or non-terminals, and β does not start with A .

2. Indirect Left Recursion

A chain of productions that eventually leads back to the same non-terminal, e.g.,

$A \rightarrow B\alpha, B \rightarrow A\beta. A \rightarrow B\alpha, B \rightarrow A\beta.$

1.2 Why Left Recursion is a Problem in Parsing

1. Top-Down Parsers Enter Infinite Loops

Recursive-descent and LL(k) parsers expand non-terminals by trying productions from left to right. If a grammar contains left recursion, the parser repeatedly expands the same non-terminal without consuming any input token, resulting in an infinite recursion.

2. Incompatibility with Predictive Parsing

LL(1) and other predictive parsers rely on a parse table built from FIRST and FOLLOW sets. Left-recursive productions make it impossible to decide uniquely which production to apply, because the FIRST set of the left-recursive production always contains the FIRST set of the non-terminal itself.

3. Performance Degradation

Even in parsers that use backtracking, left recursion causes excessive, unnecessary backtracking, leading to exponential time complexity in the worst case.

4. Parse-Table Conflicts

In LL(1) table construction, left-recursive rules produce duplicate entries, making the grammar non-LL(1).

1.3 Example

Consider the grammar for arithmetic expressions:

$$E \rightarrow E+T \mid TE \rightarrow E+T \mid T$$

A recursive-descent parser for E would immediately call $E()$ again, never advancing the input.

1.4 Solution: Elimination of Left Recursion

Direct left recursion can be removed by the following transformation:

Original:

$$A \rightarrow A\alpha \mid \beta A \rightarrow A\alpha \mid \beta$$

Transformed:

$$A \rightarrow \beta A' A' \rightarrow \alpha A' \mid \epsilon A A' \rightarrow \beta A' \rightarrow \alpha A' \mid \epsilon$$

For indirect left recursion, a systematic algorithm (such as the one by Ullman) can reorder non-terminals and apply the direct-elimination rule.

1.5 Bottom-Up Parsers and Left Recursion

Unlike top-down parsers, **bottom-up parsers (LR, LALR, etc.) can handle left-recursive grammars naturally**. In fact, left-recursive rules are often preferred for LR parsers because they produce left-associative parse trees directly.

1.6 Summary

- **Left recursion is a critical issue for top-down parsers** because it leads to infinite recursion and parse-table conflicts.
 - **It must be eliminated for LL(k) parsing** through grammar transformation.
 - **LR parsers are not affected** and can parse left-recursive grammars efficiently.
 - Understanding left recursion is essential for selecting an appropriate parsing strategy and for grammar design in compiler construction.
-

2. C++ Program: Tokenizing Arithmetic Expressions

```
cpp
#include <iostream>
#include <string>
#include <vector>
#include <cctype>
// Token types for arithmetic expressions
enum TokenType {
    INTEGER,    // Whole numbers
    PLUS,       // Addition operator '+'
    MULTIPLY,   // Multiplication operator '*'
    END         // End of input
};

// Token structure: type and lexeme
struct Token {
```

```

    TokenType type;
    std::string value;
};

std::vector<Token> tokenize(const std::string& expr) {
    std::vector<Token> tokens;
    size_t i = 0;
    size_t len = expr.length();

    while (i < len) {
        // Skip whitespace
        if (isspace(expr[i])) {
            i++;
            continue;
        }

        // Integer token
        if (isdigit(expr[i])) {
            std::string num;
            while (i < len && isdigit(expr[i])) {
                num += expr[i];
                i++;
            }
            tokens.push_back({INTEGER, num});
            continue;
        }

        // Plus operator
        if (expr[i] == '+') {
            tokens.push_back({PLUS, "+"});
            i++;
            continue;
        }

        // Multiplication operator
        if (expr[i] == '*') {
            tokens.push_back({MULTIPLY, "*"});
            i++;
        }
    }
}

```

```

        continue;
    }

    // Invalid character
    std::cerr << "Error: Unexpected character '" << expr[i]
               << "' at position " << i << std::endl;
    exit(1);
}

// Add END token to mark completion
tokens.push_back({END, ""});
return tokens;
}

void printTokens(const std::vector<Token>& tokens) {
    for (const auto& token : tokens) {
        switch (token.type) {
            case INTEGER: std::cout << "INTEGER : " << token.value << std::endl; br
eak;
            case PLUS:    std::cout << "PLUS      : " << token.value << std::endl; br
eak;
            case MULTIPLY: std::cout << "MULTIPLY : " << token.value << std::endl; br
eak;
            case END:      std::cout << "END" << std::endl; break;
        }
    }
}

int main() {
    std::string expression = "12+34*56+789";

    std::cout << "Input expression: " << expression << std::endl;
    std::cout << "\nTokenized output:\n";

    std::vector<Token> tokens = tokenize(expression);
    printTokens(tokens);
}

```

```
    return 0;  
}
```

2.1 Sample Output

Input expression: 12+34*56+789

Tokenized output:

```
INTEGER  : 12  
PLUS     : +  
INTEGER  : 34  
MULTIPLY : *  
INTEGER  : 56  
PLUS     : +  
INTEGER  : 789  
END
```

2.2 Key Features

- **Handles multi-digit integers** correctly.
- **Skips whitespace** transparently.
- **Reports invalid characters** with position information.
- **Uses an END token** to signal the end of tokenization.
- Easily extendable to support other operators, parentheses, or floating-point numbers.

3. Problem-Solving: Parse Tree Construction

3.1 Grammar

$$S \rightarrow ABA \rightarrow aA | \varepsilon B \rightarrow bB | bS \quad AB \rightarrow aA | \varepsilon \rightarrow bB | b$$

3.2 Input String

"aab""aab"

3.3 Step-by-Step Derivation

1. Start with SS :

$$S \Rightarrow ABS \Rightarrow AB$$

2. Expand AA using $A \rightarrow aAA \rightarrow aA$ (consumes first 'a'):

$$AB \Rightarrow aA \quad BAB \Rightarrow aAB$$

3. Expand the new AA again with $A \rightarrow aAA \rightarrow aA$ (consumes second 'a'):

$$aA \quad B \Rightarrow a(aA) \quad B = aaA \quad BaAB \Rightarrow a(aA)B = aaAB$$

4. Expand AA using $A \rightarrow \varepsilon A \rightarrow \varepsilon$ (no more 'a's'):

$$aaA \quad B \Rightarrow aa\varepsilon \quad B = aa \quad BaaAB \Rightarrow aa\varepsilon B = aaB$$

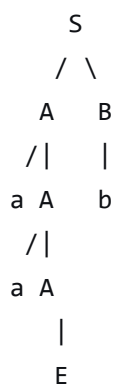
5. Expand BB using $B \rightarrow bB \rightarrow b$ (consumes the final 'b'):

$$aa \quad B \Rightarrow aa \quad b = aabaaB \Rightarrow aab = aab$$

Derivation in compact form:

$$S \Rightarrow AB \Rightarrow aAB \Rightarrow aaAB \Rightarrow aaB \Rightarrow aabS \Rightarrow AB \Rightarrow aAB \Rightarrow aaAB \Rightarrow aaB \Rightarrow aab$$

3.4 Parse Tree



Explanation

The grammar generates strings with zero or more 'a's followed by one or more 'b's.

The input "aab" has two 'a's (from A) and one 'b' (from B).

The parse tree shows how the string is derived from the start symbol S through the productions of the grammar.

The ϵ -production for A is used when no more 'a's are needed.

Notes

- This assignment covers **core syntax-analysis concepts**: left recursion, tokenization, and parse-tree construction.
- The C++ tokenizer can be extended to form the **lexical-analysis phase** of a full compiler.
- The parse-tree example illustrates how **top-down derivation** works for a simple regular-like grammar.
- Understanding these fundamentals prepares for more advanced topics: **LL/LR parsing, syntax-directed translation, and semantic analysis**.